

# JIMS-AI Complete Specification v11

Status date: June 3, 2026

## Executive Summary

JIMS-AI is a memory-centric neuro-symbolic AI runtime. It is not a single large model wrapped in a chat UI. It is a deployed system that separates durable memory, semantic retrieval, graph reasoning, symbolic validation, bounded local language-model interfaces, human review, and autonomous training orchestration.

The current implementation combines the architecture from v8, the product/runtime expansion from v9, the production deployment decisions from v10, and the real codebase state as of June 3, 2026.

## Product Identity

JIMS-AI is designed to own:

- project memory
- source-backed answers
- multi-thread user chat
- semantic retrieval
- graph reasoning context
- training workflow
- human-approved knowledge evolution
- auditable feedback and correction loops
- low-cost deployment

It should not compete with frontier models on raw general intelligence. It should compete on persistent workspace knowledge, provenance, retrieval quality, correction durability, and verifiable reasoning.

## Production Deployment

Frontend:  
Vercel Next.js

Backend:  
AWS Lambda FastAPI

Durable database:  
Supabase Postgres/Auth

Vector search:  
Cloudflare Vectorize

Graph memory:  
Neo4j Aura

Cache/session:  
Redis

Embedding and local model service:

Hugging Face Space CPU Basic

Autonomous training orchestration:  
Render web service

Offline GPU artifact work:  
Kaggle

Live services:

<https://jimsai.vercel.app>  
<https://7x27vovhmfnhcymm5ox3qiw4fy0agvcy.lambda-url.us-east-1.on.aws>  
<https://jimstechai-jimsai-embedding-service.hf.space>  
<https://jimsai-training-service.onrender.com>

## Core Architecture Decision

Do not put sentence-transformers, PyTorch, or GGUF model inference inside AWS Lambda.

Lambda remains the production API and deterministic runtime coordinator. Semantic embedding, multilingual capability classification, and bounded Qwen interfaces run in the Hugging Face Space. Render handles bounded autonomous jobs. Kaggle handles offline GPU training and artifact packaging.

## Hugging Face Local Model Stack

The Hugging Face Space hosts:

intfloat/multilingual-e5-small  
MoritzLaurer/mDeBERTa-v3-base-mnli-xnli  
ggml-org/Qwen3-1.7B-GGUF  
Qwen/Qwen3-4B-GGUF

Roles:

multilingual-e5-small:  
semantic embeddings, 768 dimensions

mDeBERTa XNLI:  
multilingual zero-shot capability classifier

Qwen3-1.7B:  
bounded JSON routing, intent fallback, math-expression normalization

Qwen3-4B:  
natural T2 rendering, ingestion extraction, canvas synthesis, invention candidates

Qwen is lazy-loaded except where startup warming is intentionally used for embedding and router readiness. The render model remains lazy because CPU loading and inference are expensive.

## Runtime Query Flow

User prompt

- > Vercel chat UI
- > Lambda /v1/query
- > Supabase session validation
- > semantic compiler
- > T1 bounded local intent interface when useful
- > L1 encoder signature
- > L2 memory indexing
- > Vectorize/Supabase hydration
- > sparse activation controller
- > v9 capability router
- > capability adapters
- > multi-index retrieval
- > abstraction engine
- > world model activation
- > reasoning bridge
- > CSSE or Qwen3-4B T2 render
- > persist thread, messages, trace, retrieval event, feedback hook
- > return Markdown response

## Capability-Intention Layer

The v11 strategy is to avoid a brittle English-only keyword router. The router combines:

- structural prompt evidence
- semantic prototype embedding similarity
- multilingual zero-shot classifier scores
- bounded Qwen JSON overlay for ambiguous cases
- fallback confidence logic

Capabilities:

- memory\_chat
- world\_knowledge
- coding
- math\_science
- creative\_text
- image\_generation
- audio\_generation
- video\_generation
- agentic\_task

The router must handle:

- chaotic prompts
- multi-intent prompts
- vague pasted content
- logs and stack traces
- code snippets
- SQL/schema/deployment prompts
- math and physics prompts
- creative writing prompts
- current-world prompts
- memory/training prompts
- non-English prompts

Routing is not truth. It selects a capability plan. Adapters, solvers, retrieval, graph context, sandbox checks, and validation decide what can actually be answered.

# Math and Science

Math and physics route to `math_science`.

Current implemented behavior:

```
bounded arithmetic executes through internal symbolic solver
supported equations execute through SymPy or fallback linear solver
Qwen3-1.7B can normalize messy multilingual math text into an expression
verified results are rendered as Markdown
failed solver attempts create explicit gaps
```

Planned extensions:

```
unit-aware physics solver
calculus adapter
statistics/probability adapter
scientific formula retrieval
paper/source retrieval for nontrivial scientific claims
```

# Coding

Coding routes include any domain humans code in:

```
Python
JavaScript
TypeScript
SQL
HTML/CSS
APIs
tests
logs
deployment
database migrations
repository architecture
debugging
package errors
infrastructure scripts
```

The correct behavior is repository-aware and test-aware. JIMS-AI should not trust generated code merely because a model produced it. It should prefer local file context, static checks, sandbox execution, tests, source references, and explicit gaps.

# Memory and Learning

JIMS-AI learns from resolved prompts through:

```
chat_threads
chat_messages
user_feedback
memory_signatures
memory_chunks
retrieval_events
retrieval_misses
```

```
training_panel_items
approval_queue
sppe_pairs
training_batches
training_artifacts
evaluation_reports
```

Feedback actions:

```
accept
correct
promote
reject
rollback
learn this
unlearn/delete memory
```

Quality-sensitive changes require human review. Autonomous jobs prepare candidates and reports; humans approve activation.

## Autonomous Training

Render is a bounded web service, not an infinite worker.

```
external cron
-> protected Render endpoint
-> one bounded unit of work
-> save progress in Supabase
-> exit
```

Render endpoints:

```
/v1/autonomous/cycle
/v1/autonomous/discover
/v1/autonomous/ingest-batch
/v1/autonomous/evaluate
/v1/autonomous/plan
/v1/autonomous/report
/v1/autonomous/kaggle/package
/v1/autonomous/reembed-hash
```

## Hash Fallback Recovery

Sentence-transformer embedding should be the normal path. Hash fallback is rare and recoverable.

If embedding fails:

```
Lambda continues with exact, structured, and graph retrieval
hash vector may be used as degraded fallback
signature metadata marks reembedding_required=true
Render /v1/autonomous/reembed-hash later replaces it with a real vector
```

# Human UI

The frontend must behave like a serious multi-thread AI workspace:

```
persistent thread sidebar
new thread
delete thread
server-backed messages
local optimistic responsiveness
Markdown rendering
feedback controls
learn/unlearn controls
training panels
review queues
artifact approval
mobile responsive layout
```

Answers should be natural and helpful, not robotic. Internal layer names and trace IDs should not appear unless the user asks for internals.

# Security

Secrets never go in public frontend variables.

Never expose:

```
Supabase service key
Hugging Face token
Render agent token
Cloudflare API token
Neo4j password
Redis URL
Kaggle token
```

Any secret pasted into chat, logs, or screenshots should be rotated.

# Comparison

| Dimension    | Frontier-only chatbot    | JIMS-AI v11                        |
|--------------|--------------------------|------------------------------------|
| Memory       | Prompt/context dependent | Supabase, Vectorize, Neo4j         |
| Retrieval    | Optional RAG             | Exact + semantic + graph           |
| Verification | Mostly model-judged      | Solver/tool/source/gap based       |
| Learning     | Weakly durable           | Feedback, SPPE, review, artifacts  |
| Deployment   | External model API heavy | Lambda + HF local models + Render  |
| Cost         | Token-scaled             | Deterministic and retrieval-first  |
| Governance   | Prompt policy            | Approval queues and artifact gates |

|         |               |                                      |
|---------|---------------|--------------------------------------|
| Chat UX | Threaded chat | Threaded chat plus training workflow |
|---------|---------------|--------------------------------------|

## Implementation Status

Implemented:

- Vercel frontend
- Lambda FastAPI backend
- Supabase schema
- multi-thread chat persistence
- feedback persistence
- learn/unlearn paths
- Hugging Face embedding service
- Hugging Face local Qwen endpoints
- multilingual capability classifier endpoint
- Render autonomous training endpoints
- Kaggle package orchestration
- Vectorize provider adapter
- Neo4j graph provider adapter
- math symbolic solver path
- capability router improvements
- Markdown CSSE rendering
- external Groq requirement removed

Remaining roadmap:

- broader scientific solver adapters
- full code sandbox verification in production workflow
- web/current-world source adapter
- artifact evaluation dashboard polish
- more mobile UI verification
- automated end-to-end browser tests
- formal retrieval-quality benchmark suite

## Final Definition

JIMS-AI v11 is a deployed, memory-centric neuro-symbolic AI operating system. It uses local models as bounded interfaces, not as the source of truth. Its durable intelligence lives in structured records, vector search, graph context, verified execution, feedback, review, and approved training artifacts.